

Redmine Chatbot測試

NL2SQL的語意解析研究



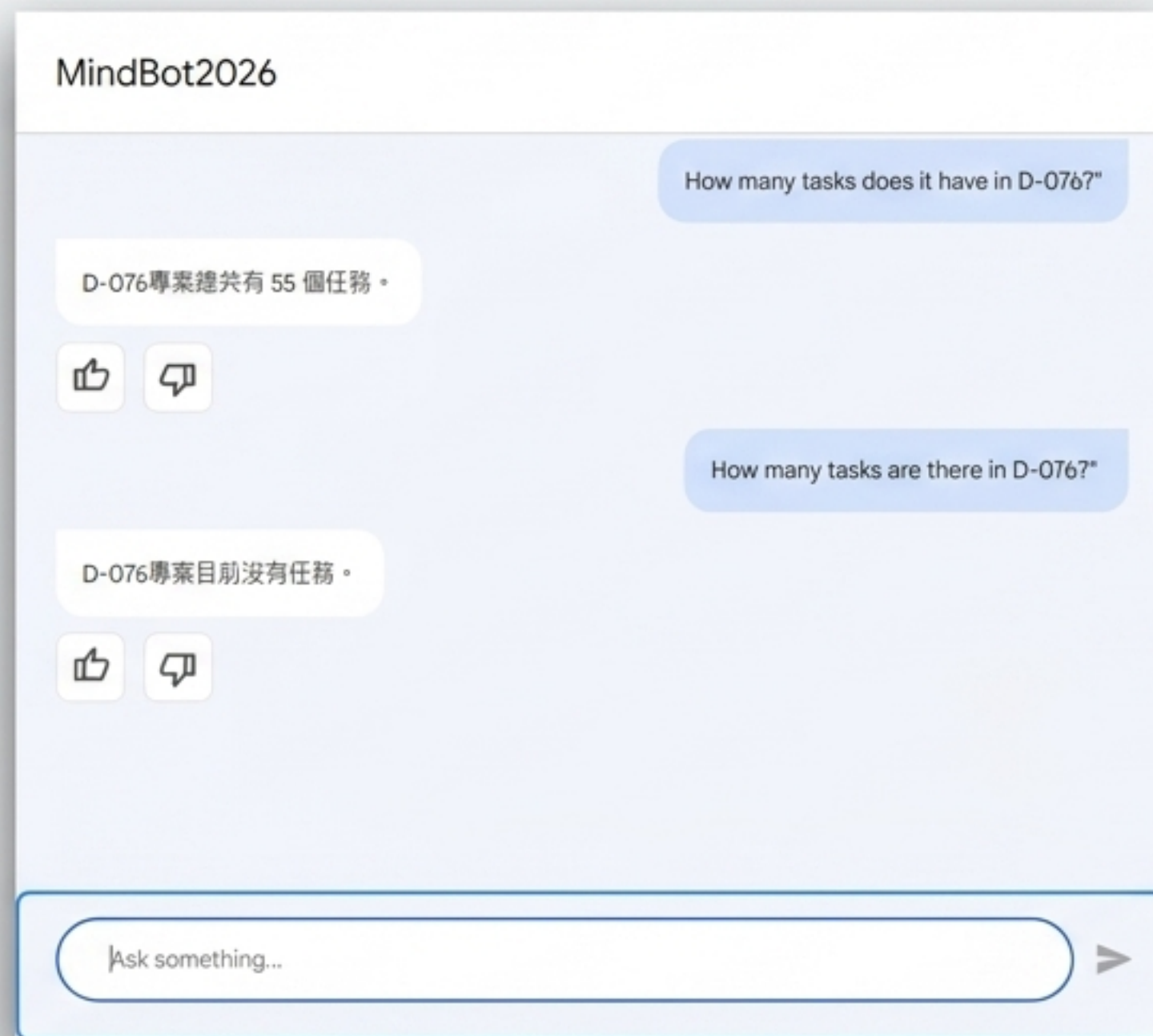
前言

本報告針對現行 NL2SQL（自然語言轉 SQL）系統在處理「同義句查詢」時出現的不一致性現象進行分析，結合 2024 年微軟最新文獻觀點，說明運用自然語言做為查詢轉換時的成效迷思。

核心問題：同義句查詢的「不一致性」

在開發以大語言模型（LLM）為核心的資料庫自然語言介面（NL2SQL）時，團隊常遭遇一項極為嚴峻的穩定性挑戰：使用者輸入兩句語義完全等價的同義句，系統卻給出截然不同的查詢結果。

這類現象在傳統 Chatbot（如純問答、客服）中較不顯著，因為原生大模型具備極強的換句話說能力。然而，一旦進入 NL2SQL 的精準數據查詢場景，大模型原生的「智慧」往往會退化，表現出極度的死板與不穩定。





句子 A (標準句型)

“How many tasks does it have in D-076?”

→ 系統成功導出 SQL，回傳正確結果 55。



句子 B (同義改述)

“How many tasks are there in D-076?”

→ 系統導出錯誤 SQL 或漏掉過濾條件，回傳結果 0。

原因探討：工程落地與速度優先下的簡化

1-3 秒 (Good UX)

商業落地實務中，使用者對 Chatbot 的耐性通常只有 **1 到 3 秒**，超過 **5 秒** 使用者體驗就會急劇下降。然而，結構完整、推理能力強的大模型（如 GPT-4o 或 Gemini 1.5 Pro）在生成 SQL 時，需要經過漫長的「Token 推理鏈」計算，耗時往往高達 5 至 10 秒以上。

5-10+ 秒 (Bad UX / Token 推理耗時)

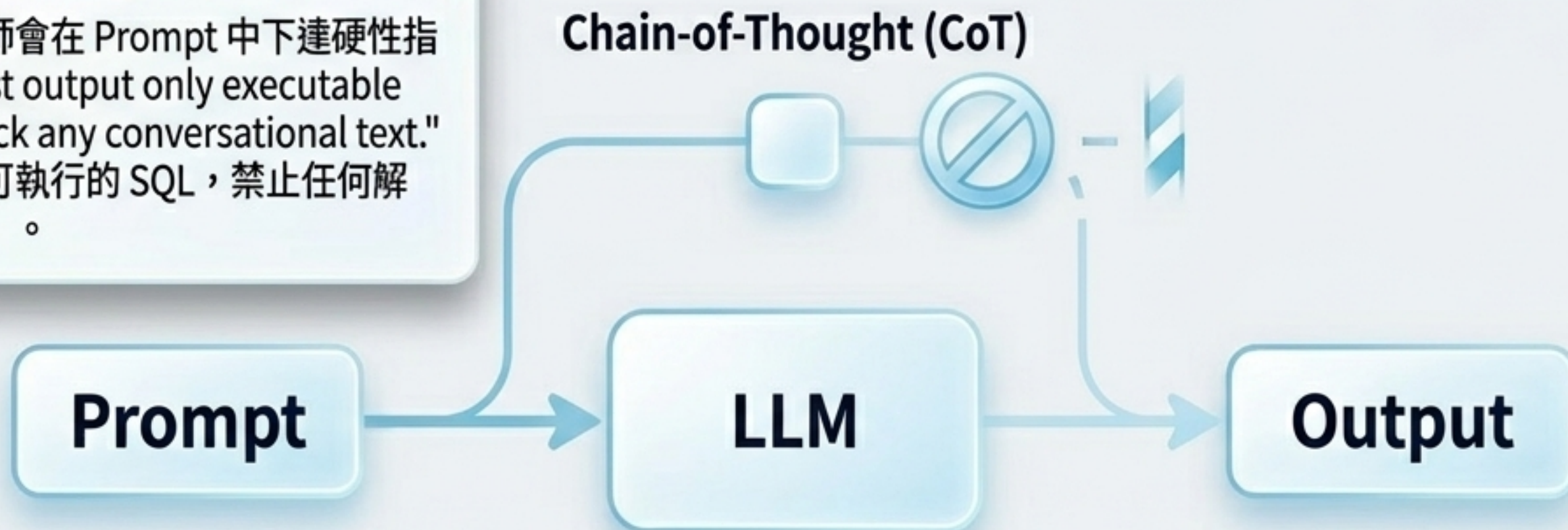
為此，NL2SQL 工具往往「**不是**」直接把話丟給原生的 Gemini，中間通常被開發團隊加上了許多「**自製的限制器**」與「**處理管線 (Pipeline)**」。

為了反應速度，系統常被迫採取「**速度優先的簡化 Pipeline 運作方式**」。

封鎖思維鏈，強迫 Token 直出

指令限制

為了快，工程師會在 Prompt 中下達硬性指令："You must output only executable SQL code, block any conversational text."
(你只能輸出可執行的 SQL，禁止任何解釋或推理過程)。



技術代價

LLM 的生成耗時與輸出 Token 數成正比。封鎖思維鏈 (Chain-of-Thought, CoT) 雖然省下了時間，卻剝奪了模型的「思考緩衝期」，強迫其進行「直覺式」的字面映射。

後果

面對“does it have”這種具備強烈語法結構特徵的句子，模型靠前向僥的機率就能寫對；但遇到需要轉個彎推理的存在句型“are there”時，解析神經就會直接跳針，漏掉關鍵條件。

論文觀點：期望和真實的落差

“

《NL2SQL is a solved problem... Not!》

微軟與 Waii 專家團隊, CIDR 2024

”

該文直言，學術榜單（如 Spider、BIRD-SQL）上的高精準度（90%+）只是溫室裡的產物，真實企業落地面臨四大技術迷思。



刷榜偏科與邏輯忽視

大廠為追求榜單第一，將模型訓練成精通 4 表 JOIN、複雜計算的「奧數選手」，卻忽略了日常對話最基本的指代消解與上下文關聯。



RAG 的斷章取義與失敗

主流框架過度依賴向量檢索（RAG）匹配 Few-shot 範例。論文測試發現，純 RAG 的效果有時甚至不如全局基準：
 $\text{RAG (18.9\%)} < \text{LLM-selected (21.1\%)} < \text{Full Schema (22.7\%)} < \text{Optimal (29.7\%)}$



指代消解的代價

在 NLP 中，搞懂 it, that, them 的難度遠高於寫 SQL。系統要求大模型一邊猜測上下文、一邊寫 SQL，兩者混雜導致模型在處理不標準句型時，解析神經直接當機。



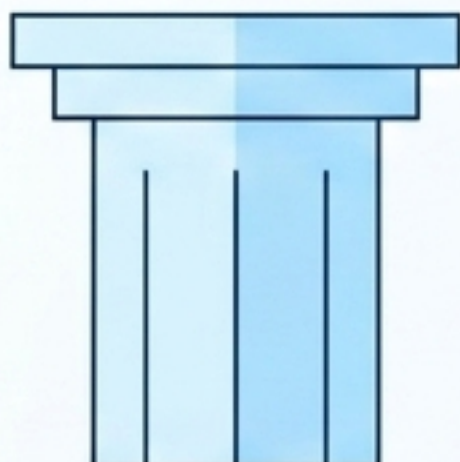
通用型與業務型的脫節

巨頭研發的模型追求「萬能」，卻極度缺乏對特定系統（如 Redmine 的特殊業務邏輯與非直覺 Schema）的深度理解。

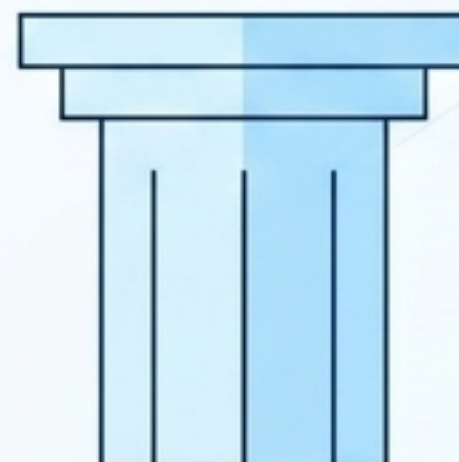
結語:改善的展望



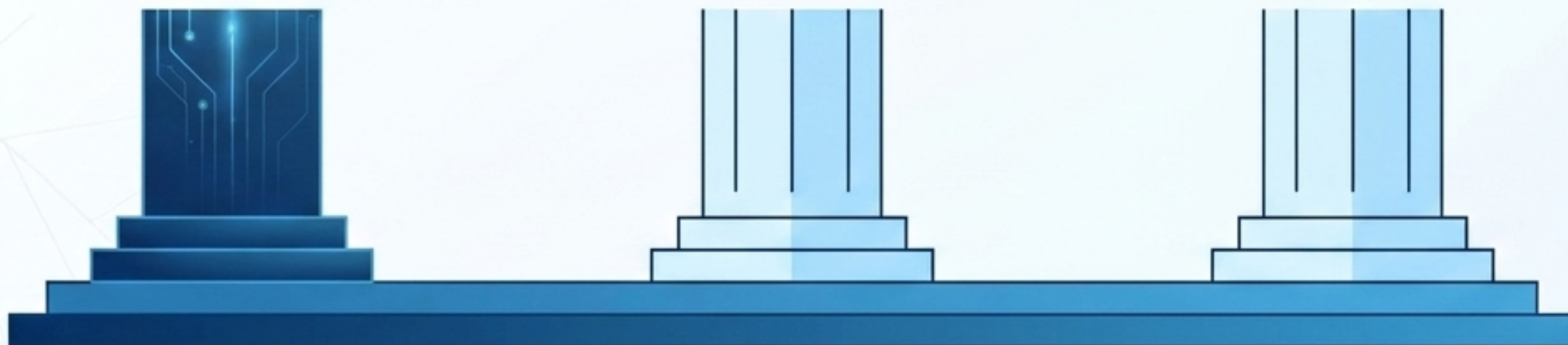
1. 外在解析代際進化
(長期技術紅利)



2. 資料結構優化
(短期治標：資料層降維)



3. 解析規則優化
(短期治標：外圍防禦)



1. 外在解析代際進化（長期技術紅利）



大腦代際升級

隨未來 GPT-4.0 / 5.1 世代普及，高併發與低延遲打破工程限制，全面釋放原生推理與指代消解能力。

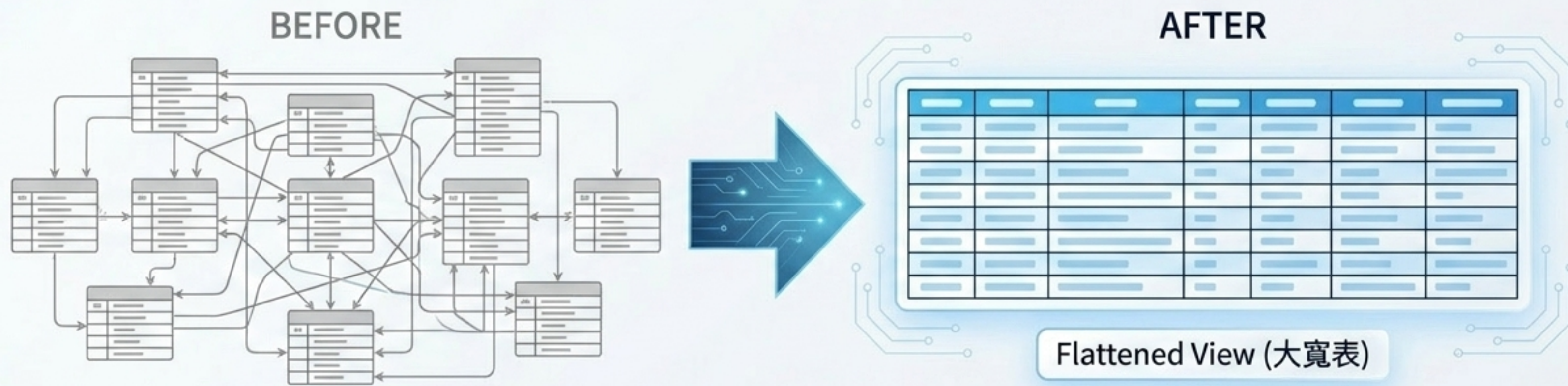


高維語意收斂

利用新一代模型的意圖對齊能力，天然消除「存在句（are there）」與「標準控制句（does it have）」的語意分歧。



2. 資料結構優化（短期治標：資料層降維）



展平大寬表（Flatten）

於 BigQuery 預先建立 AI 專用檢視表（View），將 Redmine 複雜的多表 JOIN 降維為「單表 WHERE 過濾」，釋放小模型算力。

元數據（Metadata）注入

在欄位補齊詳細的 Description 註釋，動態隨 Prompt 餵給模型，快速補足小模型缺乏的業務常識。

3. 解析規則優化（短期治標：外圍防禦）

前端語義歸一化

增設 < 0.05秒的輕量改寫字典（例：偵測到 are there 自動改寫為標準的 does it have），在外圍直接消滅分歧。

多元 Few-shot 制定

豐富少樣本提示範例，同時納入「擁有句」與「存在句」的 SQL 對照組，拓寬模型語意邊界。

動態 Prompt 護欄

強制注入 LOWER() 函數消除大小寫敏感問題；建立「實體提取規則（如 D-XXX）」，確保核心過濾條件絕不遺失。